



1만 대 IoT 디바이스 헬스 모니터링 플랫폼

[기술 스택]

Spring Boot 3 · Redis · PostgreSQL · TimescaleDB · Kafka · Resilience4j · Grafana/Loki/Prometheus · Sentry



Executive Summary

“1만 대 IoT 디바이스의 10초 주기 하트비트를 처리하는 헬스 모니터링 플랫폼”

- **[Problem]** 도입 초기 에러율 14.75%로 실시간 상태 파악이 불가능했으며, Redis 장애 1회로 Sentry 월 쿼터가 전소되는 관측성 단절 경험
- **[Impact]** Circuit Breaker 기반 동적 Fallback 파이프라인 구축으로 에러율 0%·P95 370배 개선, 특정 인프라 장애에 서도 무중단 운영 달성



3-Phase 핵심 해결 전략

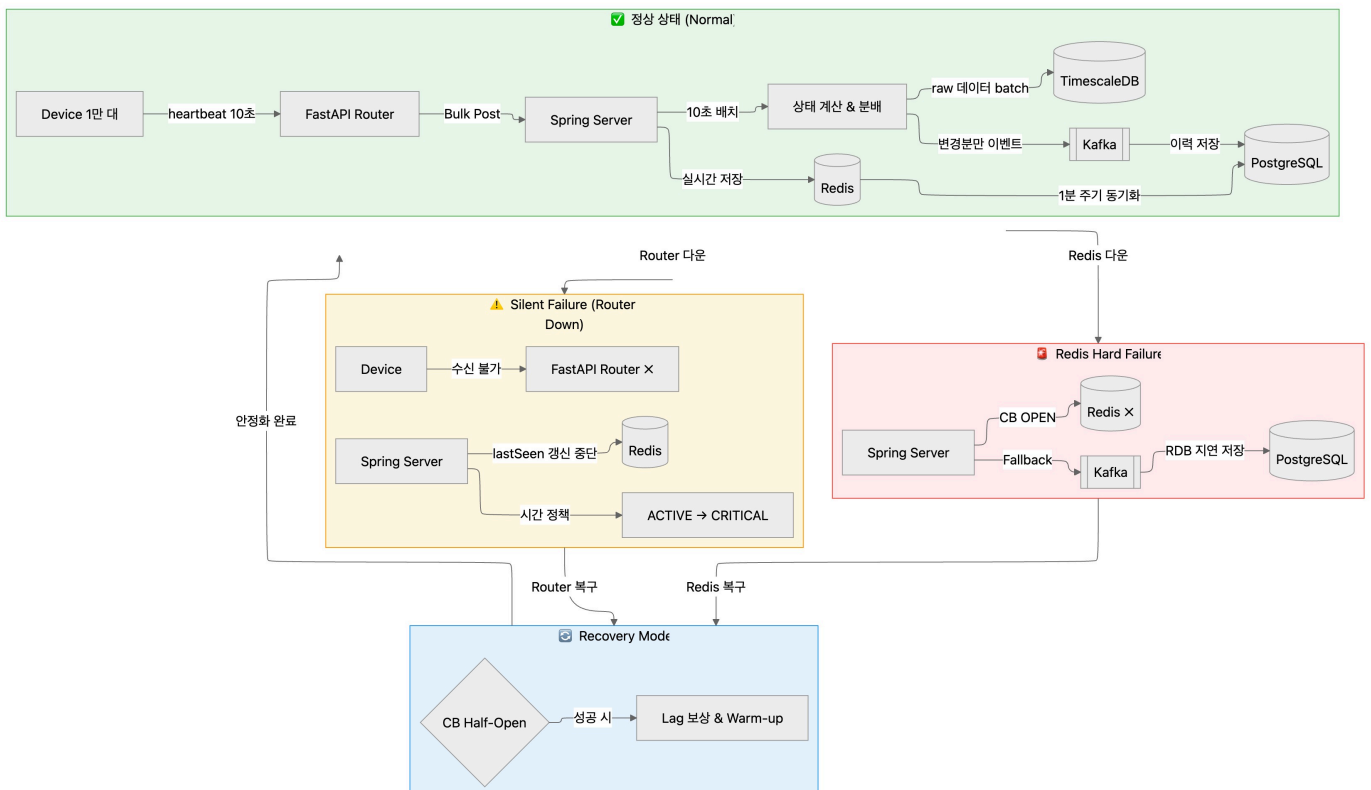
Phase	핵심 문제	해결 전략	주요 성과 (Impact)
Phase 1: 성능	RDB 커넥션 고갈 및 Deadlock (P95: 20s)	Redis Write-Behind + VT 병렬화	- P95 속도 370배 개선 - DB I/O 96.6% 감소
Phase 2: 장애 대응	Redis SPOF, Silent Failure	CB + Recovery Mode + Kafka를 통한 비동기 상태 전이 이벤트	- 카오스 테스트 전 시나리오 에러율 0.00% 유지
Phase 3: 관측성	장애 시 로그 폭발 및 Sentry 쿼터 소진	CB 전이 전송 + Log Tiering	- Sentry 알림 폭탄 → CB 전이 4건으로 압축 (장애 1건당) - 로그 비용 \$0 유지



장애 시나리오별 동적 Fallback을 활용한 서비스 무중단 설계

[데이터 중심의 저장소 전략]

- **Redis:** 초고성능 하트비트 수신 및 최신 상태 관리 (성능)
- **PostgreSQL:** 최종 상태값·서비스 메타데이터·상태 변경 이력 (안정성)
- **TimescaleDB:** 대량의 하트비트 이력 및 통계 분석 (확장성)
- **Kafka:** 비동기 이벤트 및 장애 완충 파이프라인 (내구성)



Phase 1: 성능 최적화

프로젝트 개요

- 10,000대 IoT 디바이스의 연결 상태를 실시간 관제하는 헬스체크 시스템. 기기당 10초 주기 하트비트
- → 초당 1,000건(1,000 TPS) 쓰기 트래픽 처리가 핵심 요구사항

핵심 문제

- 기존 RDB 직결 구조로는 고빈도 쓰기 트래픽을 감당 불가
- Connection Pool 고갈 및 Deadlock → 에러율 14.75%, P95 20초
- 소프트웨어 튜닝만으로는 극복 불가한 하드웨어 리소스 한계

Write Path 개선 과정

시도	결과
Bulk API 도입	Deadlock 즉시 발생
Sorting 적용	Deadlock 감소, 성능 미개선 (P99 9.76s)
RDB 한계 확인	HikariCP 고갈 / Disk I/O 병목 근본 원인 확인
Redis Write-Behind 전환	쓰기 격리 완성, 에러율 0%

데이터 처리 파이프라인 (Data Pipeline Lifecycle)

"트래픽 성격에 따른 주기적 배치 처리를 통해 DB 부하 96% 절감"

- 상시 (Real-time): 하트비트 수신(Redis 캐싱) 및 Kafka Consumer 이벤트 처리
- 10초 주기 (Near Real-time): 1만 대 상태(On/Off) 계산, TimescaleDB 적재, 상태 변경 이벤트(Kafka) 발행
- 1분 주기 (Batch): Redis 최신 상태 RDB(PostgreSQL) 동기화 및 자체 스냅샷(BGSAVE) 백업

쓰기 최적화 (Write Path)

- 10초당 1만대의 헬스체크 저장 시 RDB Connection Pool 고갈 및 Deadlock(에러율 14.75%)이 발생
- Bulk Update, Sorting 등 소프트웨어 튜닝으로는 디스크 I/O의 물리적 한계를 극복 불가

해결: Redis Write-Behind 도입. 평시에는 Redis에만 Overwrite, 1분 주기로 RDB에 비동기 배치 업데이트

지표	개선 전 (RDB Bulk)	개선 후 (Write-Behind)	Impact
에러율	14.75% (Deadlock)	0.00%	장애 완전 해소
P95 Latency	20.11s	54.28ms	약 370배 개선
DB I/O	10초당 10,000회 UPDATE	1분당 1회 배치	96.6% 감소

조회 최적화 (Read Path)

- 1만 대 상태를 10초 배치로 조회할 때, 디바이스별 개별 Redis 호출(N+1)로 p50 9.12s가 발생
- 서비스 그룹 단위 HGETALL 일괄 조회로 Redis 왕복을 1,000회로 줄였고(p50 6.44s), Virtual Thread 1,000개 병렬화로 p50 210ms, p99 2.71s를 달성

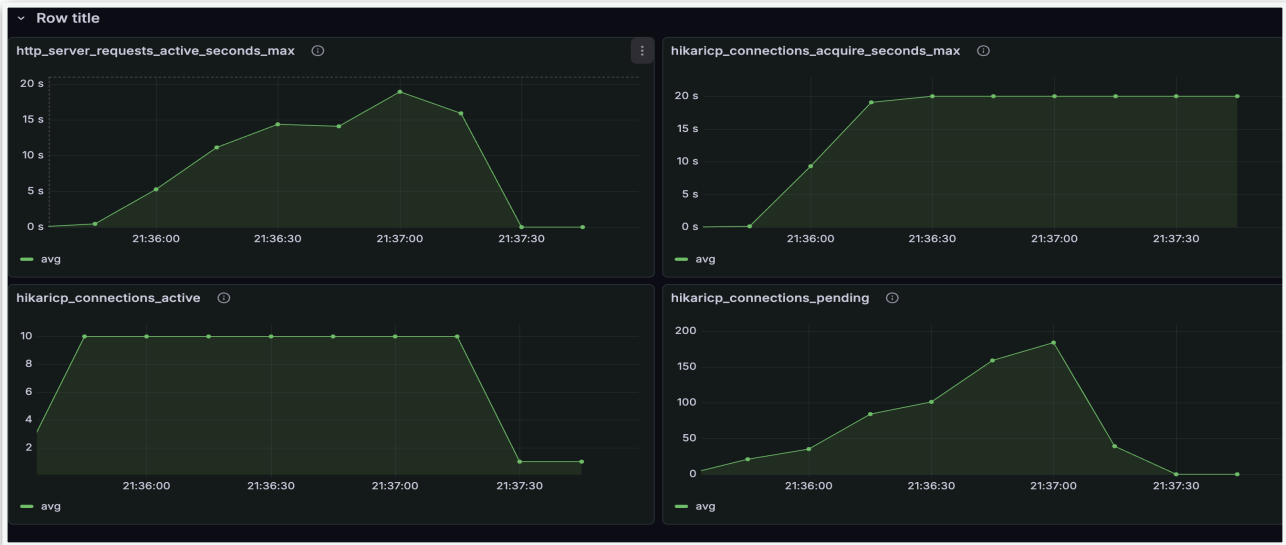
 Virtual Thread 병렬화 도입 사유: 단순 조회 최적화(Pipeline)를 넘어,

[Redis 조회 → 상태 계산(CPU) → Kafka 발행]에 이르는 전체 사이클을 그룹 단위로 병렬 처리하기 위해 적용

단계	변경	Redis p50	Total p99
초기 (N+1)	HGET × 10,000	9.12s	30s(timeout)
HGETALL 일괄 조회	HGETALL × 1,000	6.44s	15.7s
Virtual Thread 병렬화	VT 1,000개 동시 처리	210ms	2.71s

[Before]

RDB 커넥션 풀 고갈 및 100% 포화 상태 확인 (최대 대기 시간 20s 돌파로 인한 실질적 서비스 마비)



[After]

Redis기반 I/O 격리를 통해 응답 속도 370배 개선 및 DB 부하 제거



🔗 Phase 2로의 브릿지

성능은 확보했으나 Redis가 SPOF가 됐다. 장애 시 최대 1분간 디바이스 상태를 모르는 것은 IoT 모니터링에서 허용 불가



Phase 2: 장애 대응 설계 — Hybrid Resilience

I 성능 구간(Redis)과 안정성 구간(RDB)을 분리. 평시에는 Redis로 빠르게, 장애 시에는 즉시 RDB Fallback



시나리오 1: Redis Hard Failure

Redis 다운 시 HGETALL 타임아웃 대기 → 1,000 서비스 x 타임아웃 = 스레드 고갈. Resilience4j Circuit Breaker로 자동 감지 후 RDB Fallback으로 즉시 전환

라이브러리 선택: Resilience4j는 Spring Boot 3 공식 지원하며, 데코레이터 패턴으로 기존 코드 수정 최소화

[CB 설정값과 근거 (Grafana 실측 기반)]

설정	값	근거
slow-call-duration	300ms	Redis p50(200ms)의 1.5배
slow-call-rate	80%	window 중 80% 초과 시 OPEN. 간헐적 지연 허용
timeout (Lettuce)	800ms	10초 배치 주기 역산. 초과 시 failure count 반영
failure-rate	50%	window 중 50% 실패 시 OPEN
wait-duration	10s	배치 주기와 일치. 1사이클 대기 후 복구 시도

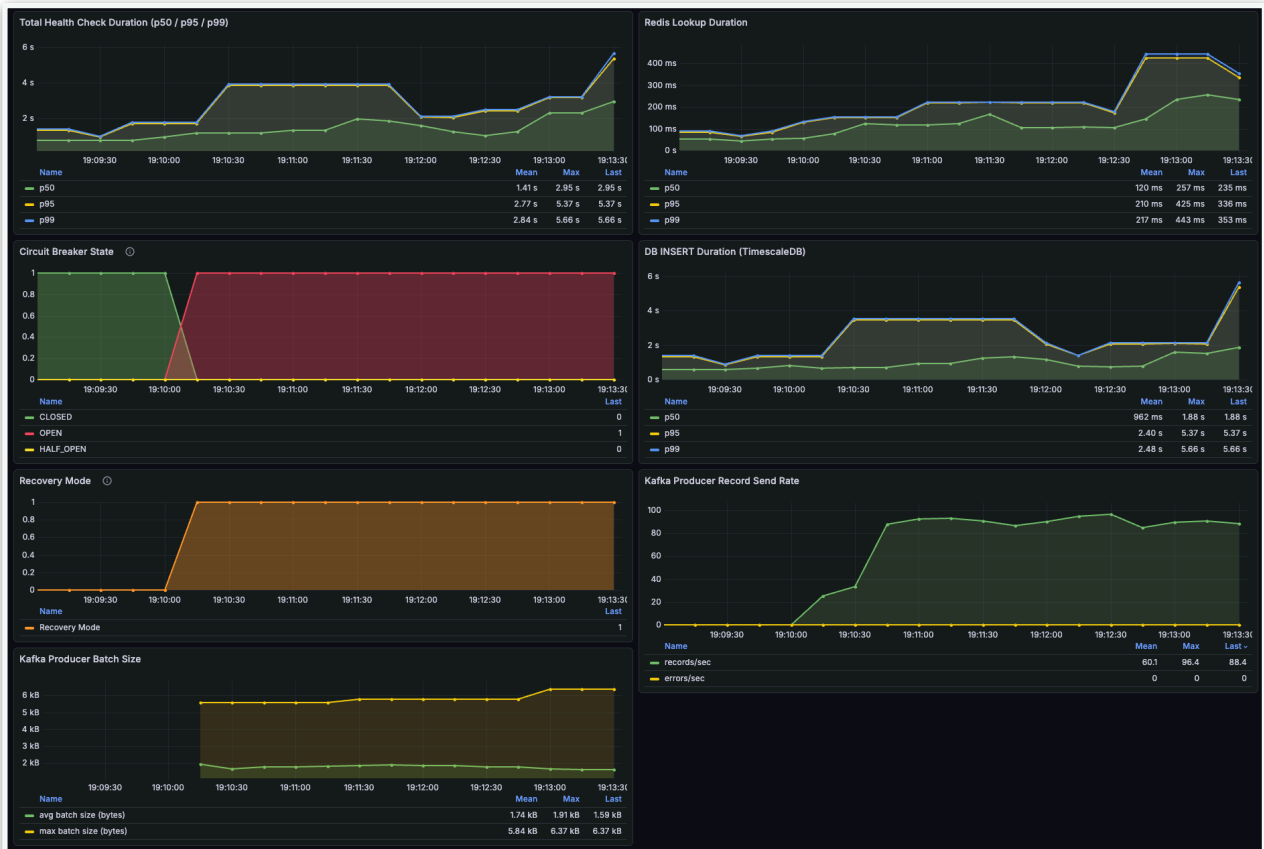
300ms ~ 800ms 회색 지대: 300ms 초과는 slow-call로 누적 카운팅, 800ms 초과만 즉시 failure 처리

일시적 네트워크 지터를 장애로 오판하지 않기 위한 실무적 방어 로직

Fallback: CB OPEN 시 Redis 호출 즉시 차단(대기 0ms)

읽기는 JPA 엔티티의 getStatus() 재활용(추가 쿼리 없음), 쓰기는 RDB 직접 저장

[카오스 테스트: docker stop redis]



지표	Baseline	장애 시	분석
CB 상태	CLOSED	OPEN	즉시 전환
Kafka records/sec	0	88	상태 차이로 전이 이벤트 발생
DB INSERT p50	612ms	962ms	RDB에 부하 집중
에러율	0%	0%	서비스 무중단 Fallback 성공



시나리오 2: Silent Failure

Redis도 Spring도 정상인데 heartbeat 유입이 멈춤. 예외가 발생하지 않으므로 CB Close

설계 원칙: "원인을 알 수 없다"는 한계를 인정

클라우드 서버는 디바이스 장애인지, 라우터 장애인지, 네트워크 단절인지 구분 불가

원인 불문 시간 정책으로 상태를 전이하되, Recovery 시점에서 오탐을 방어함

경과 시간	상태 전이	설명
0 ~ 15초	ACTIVE	정상 (10초 주기 + 버퍼 5초)
15 ~ 25초	WARNING	1회 누락
25초 이상	CRITICAL	2회 연속 누락
Recovery Mode	임계값 + 60초	lagCompensation 오탐 방지

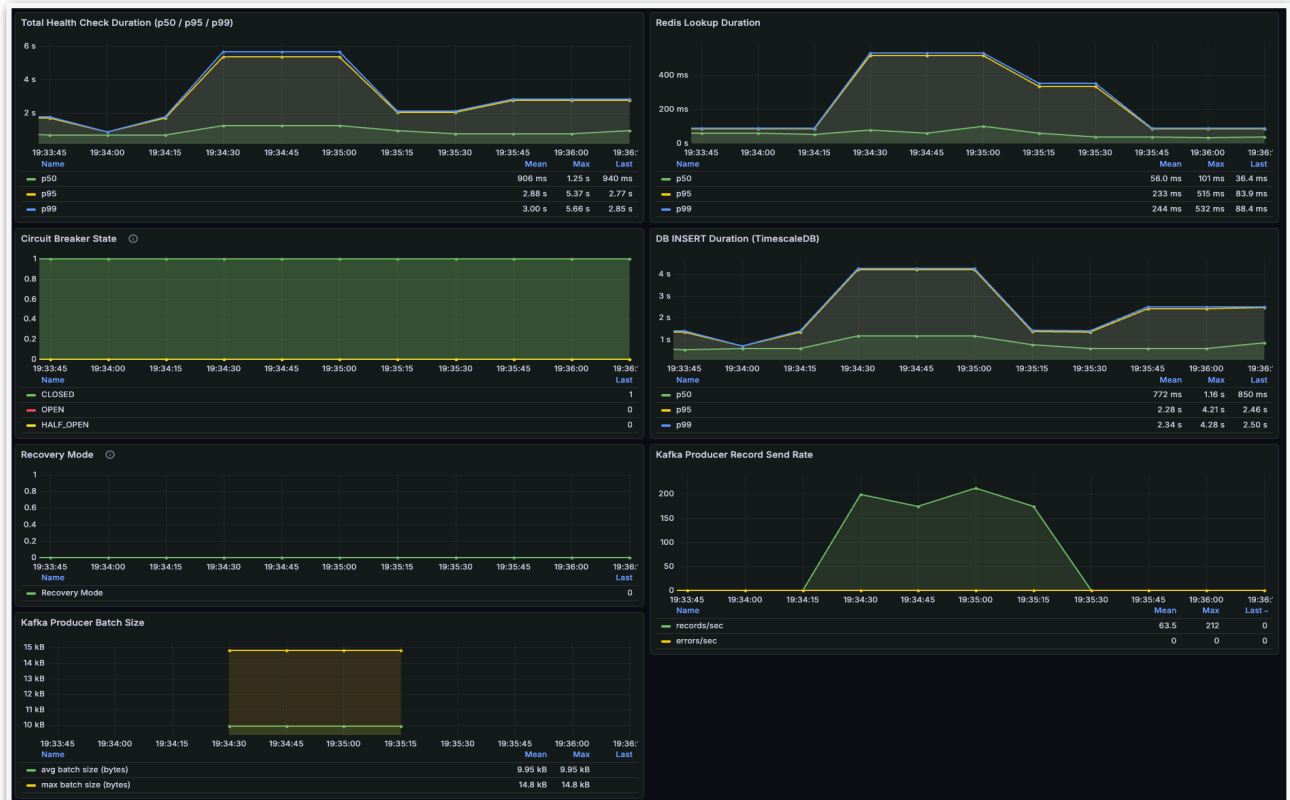
Event Storm 방지: Kafka 완충 지대(Buffer)

1만 대 동시 전이 시 최대 423 events/sec 발생. DB에 직접 쏘면 연쇄 다운(카오스 테스트에서 Postgres CPU 88% 확인). Kafka로 비동기 발행하여 헬스체크 사이클과 이벤트를 시간적으로 분리(Temporal Decoupling)

- **Producer:** 변경분만 Kafka에 비동기 발행, 즉시 반환
- **Consumer:** 자체 속도로 DB에 이력 저장, DB 과부하 방지

연쇄 장애 발견: 카오스 테스트에서 Silent Failure가 Event Storm → Redis 과부하(p95: 100ms→2.06s) → CB 연쇄 트리거까지 유발하는 복합 장애임을 확인

[카오스 테스트 : 디바이스 헬스체크 중단]



지표	Baseline	Silent Failure	분석
Kafka records/sec	0	423	시나리오 1(88)의 4.8배
Postgres CPU	24%	88%	이벤트 폭발 → DB 과부하
Redis Lookup p95	100ms	2.06s	Event Storm → Redis 과부하
CB 상태	CLOSED	OPEN (연쇄)	Redis 과부하 → CB 연쇄 트리거

향후 개선: 상태 전이 이벤트에 Rate Limiting/Debouncing을 적용하여 서비스 그룹 단위 집계 후 1건 발행 예정

시나리오 3: Recovery

장애 복구 직후 바로 정상 모드로 전환하면 안 되는 이유:

1. **Stale 데이터**: DB 데이터가 최대 60초 stale (Sync 1분 주기)
2. **트래픽 폭주**: 밀려있던 헬스체크가 한꺼번에 유입
3. **재과부하**: CB HALF_OPEN에서 전체 트래픽 개방 시 Redis 재과부하

Recovery Mode : "장애 이후의 장애"를 예방하는 설계

단계	설정	근거
Warm-up	40초 (3사이클 × 10초 + 버퍼 10초)	- CB HALF_OPEN에서 3회 성공해야 CLOSED - 불확실성 흡수
lagCompensation	60초	- SyncScheduler 1분 주기와 일치 - CRITICAL 임계값을 85초(25+60)로 완화

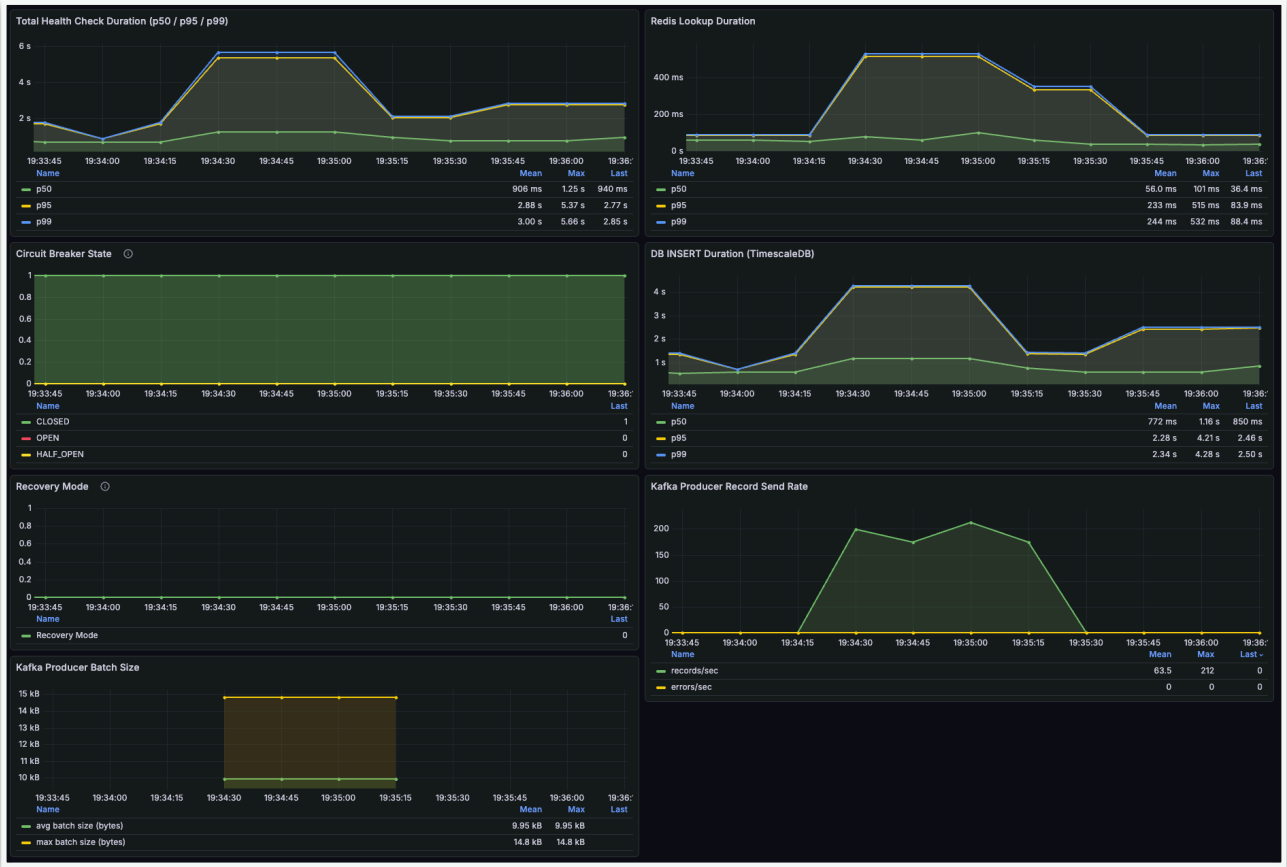
[카오스 테스트: docker start redis]



지표	장애 중	Recovery 중	정상 복구
CB 상태	OPEN	HALF_OPEN → CLOSED	CLOSED
Recovery Mode	1	1 (60초 유지)	0
Kafka records/sec	88	93 → 수렴	0

I 핵심: CB CLOSED 후에도 Recovery Mode가 60초간 유지되어 오탐 발생 없이 점진적 복구 확인

Silent Failure Recovery (디바이스 헬스체크 재시작)



지표	Silent Failure 중	Recovery 중	정상 복귀
Kafka records/sec	423	212 → 수렴	0
Postgres CPU	88%	50%	18.7%



Phase 3: 관측성 - 비용 제약 안에서의 아키텍처

I 장애 추적과 SaaS의 무료 쿼터(Sentry 5,000건, Loki 50GB)와 클라우드 유지 비용을 방어하는 관측성 파이프라인을 구축



1. 로그 이중화 및 수명 주기 최적화 (Log Tiering)

1만 대의 디바이스가 10초 배치로 쏟아내는 DEBUG 상태 변경 로그를 Grafana Cloud로 모두 전송할 경우, 하루 만에 수 GB가 발생하여 월 무료 쿼터(50GB)를 즉시 초과

이를 방어하기 위해 로그의 목적과 용량에 맞춰 저장 경로를 이중화

경로 (온도)	로그 레벨	보존 기간	용도 및 비용 전략
Loki (Hot)	INFO 이상	14일	- 실시간 모니터링 및 대시보드 쿼리. - 월 50GB 내에서 \$0로 운영 통제
FILE (Warm)	DEBUG 포함 전체	7일(롤링)	- 장애 사후 상세 분석 - EC2 로컬 디스크 활용
S3 / Glacier (Cold)	DEBUG 포함 전체	90일~365일	- 컴플라이언스 및 장기 보관 - EC2 Cron으로 S3 전송 후 Glacier 자동 전환(Lifecycle)



장애 분석 흐름 (Troubleshooting Flow)

- 발견:** Sentry 알람 또는 Loki(실시간 INFO)에서 장애 발생 시간대와 traceId 확인
- 추적:** 상세 원인 규명이 필요할 때만, 해당 일자의 S3/로컬 FILE 로그를 다운로드하여 grep traceId로 DEBUG 레벨까지 심층 분석



2. Log-based Tracing (MDC 다차원 컨텍스트 주입)

배치 스케줄러는 HTTP 컨텍스트가 없어 traceId가 자동 생성되지 않음

- 해결:** @Observed로 traceId를 주입, 비즈니스 컨텍스트(servicId, cbState, recoveryMode)를 **MDC**에 담아 Loki에서 다차원 필터링이 가능하도록 구성
- Virtual Thread 전파 이슈:** MDC는 ThreadLocal 기반이라 VT 컨텍스트 스위칭 시 초기화, MDC.getCopyOfContextMap()으로 스케줄러 스레드에서 캡처 후 각 VT 시작 시 복원하여 1,000개 VT 로그를 단일 traceId로 묶음



3. Sentry: CB 전이 이벤트만 전송

1. Sentry 월 쿼터(5k) '1분 소진' 방지

- 문제:** Redis 장애 시 **분당 6,000건 에러 폭주 → 월 쿼터 1분 만에 소진**
- 해결:** 일반 예외 전송을 차단하고, 핵심 지표인 **'상태 전이'**만 선별 전송

2. 중복 알림 제어 (Troubleshooting)

- 현상:** 복구 실패 시 HALF_OPEN → OPEN 로그 수십 건 반복 발행
- 기술적 해결:** volatile boolean 플래그로 첫 **1회 실패만 전송** (정상 복구 시 리셋)
- 성과:** **알림 노이즈 99% 제거** 및 온콜 피로도 개선

전이	레벨	근거
CLOSED → OPEN	ERROR	장애 시작
OPEN → HALF_OPEN	INFO	복구 시도
HALF_OPEN → CLOSED	INFO	복구 완료
HALF_OPEN → OPEN	WARNING	복구 실패 (첫 1회만 전송)

[Sentry 분석: 자가 치유(Self-healing) 타임라인 확인]

"장애 인지부터 자동 복구까지의 Self-Healing 전 과정 검증"

장애 발생 후 800ms 내 즉각 차단 및 Fallback 전환을 데이터로 증명

- 14:00:01 **CLOSED → OPEN [ERROR]:** 장애 감지 및 즉각 차단 (장애 시작)
- 14:00:11 **OPEN → HALF_OPEN [INFO]:** 시스템 자동 복구 시도 시작
- 14:00:12 **HALF_OPEN → OPEN [WARNING]:** 복구 실패 확인 (1회 요약 알림)
- 14:03:41 **OPEN → HALF_OPEN [INFO]:** 재시도 및 연결 확인
- 14:03:42 **HALF_OPEN → CLOSED [INFO]:** 자가 치유 완료 및 정상 모드 복구

Feed

iot-edge ▾ All Envs ▾ 14D ▾

Last Seen ▾ Save As

Q is unresolved x

Issue	Last Seen	Age	Trend	24h 14d	Events	Users	Assignee
Circuit Breaker OPEN_TO_HALF_OPEN: ... (No error message) IOT-EDGE-B	28s ago	4min	2 New		8	1	
Circuit Breaker HALF_OPEN_TO_OPEN: ... (No error message) IOT-EDGE-D	2min ago	2min	1 New		1	1	
Circuit Breaker HALF_OPEN_TO_CLOSE... (No error message) IOT-EDGE-C	4min ago	4min	1 New		1	1	
Circuit Breaker CLOSED_TO_OPEN: Redi... (No error message) IOT-EDGE-A	4min ago	4min	1 New		1	1	

1-4 of 4 < >

Issues / IOT-EDGE-A

Circuit Breaker CLOSED_TO_OPEN: Redis 헬스체크 [redis-health-monitor]
| (No error message)
New

Events (total) 1 Users (90d) 1

Resolve ▾ Archive ▾ ...

Priority ▾ Assignee

Unassigned ▾

All Envs ▾

Since First Seen ▾

Q Filter events...

Event 1

User 1

Mar 5 6:00 AMMar 5 7:00 AMMar 5 8:00 AM

user

environment

level

runtime.name

View all tags

100% 218.159.228.243

100% dev

100% error

100% Amazon.com Inc.

Events ▾ in this issue < > First Latest Recommended View More Events Copy as ▾

ID: 20b0c0b0 5 minutes ago JSON Jump to: Highlights Stack Trace Trace Tags Context

218.159.228.243 dev

Highlights Edit

handled --

level error

transaction --

url --

Trace: Trace ID

9c7e11e61c3647c79bb1fc72cadba119

Message
Circuit Breaker CLOSED_TO_OPEN: Redis 헬스체크 [redis-health-monitor]



카오스 테스트 종합

시나리오	CB	Recovery	Kafka Peak	Errors/sec
정상 운용	CLOSED	0	0	0
Redis Hard Failure	OPEN	1	88	0
Redis Recovery	CLOSED	1→0	수렴	0
Silent Failure	CLOSED→OPEN	0	423	0
Silent Recovery	CLOSED	0	212→0	0

I 어떤 인프라 장애 상황에서도 유저 에러율 0% 유지 (무중단 보장)



회고

1. 기술적 트레이드오프: 정합성 vs 가용성

- Write-Behind으로 최대 60초의 데이터 지연 리스크가 발생하지만, lagCompensation(60초)으로 오탐을 방지
- '1만 대 기기의 실시간 관제'라는 서비스 목적에 따라 **시스템 가용성**을 최우선 가치로 설계

2. 설계의 지속 가능성: Trade-off 기반 아키텍처 결정

[복잡도 vs 안정성]

- Circuit Breaker와 Recovery Mode 도입으로 시스템 복잡도는 증가, 특정 저장소(Redis)의 장애가 전체 시스템으로 전파되는 연쇄 장애를 완벽히 차단하는 구조 확립

[비용 vs 확장성]

- 향후 디바이스 대수 증가 시, 고가의 RDB Scale-up 비용을 지불하는 대신 Redis 중심의 수평 확장(Scale-out)만으로 트래픽을 소화할 수 있는 저비용 아키텍처 확보

3. 데이터 중심의 저장소 전략 (Right Tool for the Right Job)

- 데이터의 성격과 주기, 접근 빈도에 따라 저장소 역할을 명확히 분리
- 상세 내용은 1페이지 아키텍처 다이어그램 참조